Light

GCSE Computer Science – Searching and Sorting Algorithms (Set 1)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • AQA 8525 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Algorithms Searching Sorting

Multiple-choice topic test for **GCSE Computer Science (AQA 8525)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

What is a linear search?

- ○ **A.** An algorithm that checks each item in a list one by one from the start until the target is found or the end is reached
- ○ **B.** An algorithm that repeatedly divides a list in half to find an item
- ○ **C.** An algorithm that sorts items before searching
- ○ **D.** An algorithm that only works on numbered lists

Q2 (1 mark)

What is a binary search?

- ○ **A.** An algorithm that checks every item in order
- ○ **B.** An algorithm that only searches through two items
- ○ **C.** An algorithm that converts items to binary before searching
- ○ **D.** An algorithm that repeatedly divides a sorted list in half, comparing the middle item to the target to eliminate half the remaining items each time

Q3 (1 mark)

What is bubble sort?

- ○ **A.** A sorting algorithm that divides the list into smaller sublists
- ○ **B.** A sorting algorithm that repeatedly compares adjacent pairs of items and swaps them if they are in the wrong order
- ○ **C.** A sorting algorithm that finds the smallest item and moves it to the front
- ○ **D.** A sorting algorithm that only works with numbers

Q4 (1 mark)

What is the basic approach used by merge sort?

- ○ **A.** Compare adjacent items and swap if needed
- ○ **B.** Find the largest item and move it to the end
- ○ **C.** Divide the list into halves repeatedly until single items remain, then merge the parts back together in sorted order

- ○ **D.** Randomly shuffle items until they are sorted

Q5 (1 mark)

What condition must be met before a binary search can be performed?

- ○ **A.** The list must contain only numbers
- ○ **B.** The list must have an odd number of items
- ○ **C.** The list must be stored in an array
- ○ **D.** The list must be sorted

Q6 (1 mark)

Where does a linear search begin looking for an item?

- ○ **A.** At the first item in the list
- ○ **B.** At the middle of the list
- ○ **C.** At the end of the list
- ○ **D.** At a random position

Q7 (1 mark)

During one pass of bubble sort on a list being sorted in ascending order, what happens?

- ○ **A.** The smallest item moves to the start of the list

- ○ **B.** All items are placed in their final positions
- ○ **C.** The list is divided into two halves
- ○ **D.** Adjacent pairs are compared and swapped if the first is larger than the second

Q8 (1 mark)

Why is merge sort described as using a 'divide and conquer' approach?

- ○ **A.** Because it splits the problem into smaller parts, solves each part, then combines the solutions
- ○ **B.** Because it compares items two at a time
- ○ **C.** Because it conquers the largest items first
- ○ **D.** Because it divides items into odd and even positions

Q9 (1 mark)

Which searching algorithm can be used on an unsorted list without any preparation?

- ○ **A.** Binary search
- ○ **B.** Both linear and binary search
- ○ **C.** Neither linear nor binary search
- ○ **D.** Linear search

Q10 (1 mark)

How does bubble sort know when the list is fully sorted?

- ○ **A.** When it has completed exactly n passes, where n is the number of items
- ○ **B.** When the first item is the smallest
- ○ **C.** When no swaps are made during a complete pass through the list
- ○ **D.** When half the items have been checked

Q11 (1 mark)

In binary search, what happens to the search area after each comparison with the middle item?

- ○ **A.** It stays the same size
- ○ **B.** It is approximately halved
- ○ **C.** It reduces by one item
- ○ **D.** It doubles in size

Q12 (1 mark)

In merge sort, what is the purpose of the merge step?

- ○ **A.** To split the list into individual items
- ○ **B.** To combine two sorted sublists into a single sorted list by comparing items from each

- ○ **C.** To randomly arrange items
- ○ **D.** To remove duplicate items from the list

Q13 (2 marks)

A linear search is performed on the list `[8, 3, 12, 5, 9]` to find the value 5. How many comparisons are made before the value is found?

- ○ **A.** 1 comparison
- ○ **B.** 2 comparisons
- ○ **C.** 3 comparisons
- ○ **D.** 4 comparisons

Q14 (2 marks)

A binary search is performed on the sorted list `[2, 5, 8, 12, 16, 23, 38]` to find the value 23. Which item is compared first?

- ○ **A.** 12 (the middle item)
- ○ **B.** 2 (the first item)
- ○ **C.** 38 (the last item)
- ○ **D.** 23 (the target item)

Q15 (2 marks)

The list `[6, 2, 8, 1]` undergoes one pass of bubble sort in ascending order. What is the list after this first pass?

- **A.** [1, 2, 6, 8]
- **B.** [2, 1, 6, 8]
- **C.** [2, 6, 1, 8]
- **D.** [6, 2, 1, 8]

Q16 (2 marks)

A sorted list contains 1000 items. Approximately how many comparisons might be needed in the worst case for linear search versus binary search to find an item?

- **A.** Linear: 10, Binary: 1000
- **B.** Linear: 500, Binary: 500
- **C.** Linear: 1000, Binary: 1000
- **D.** Linear: 1000, Binary: 10

Q17 (2 marks)

In which situation would linear search be a more practical choice than binary search?

- **A.** When searching a list of 10,000 sorted items repeatedly
- **B.** When the search must be completed as quickly as possible on large data
- **C.** When searching a small unsorted list only once
- **D.** When the data is already sorted in descending order

Q18 (2 marks)

How many swaps are performed in total when bubble sort fully sorts the list `[4, 3, 2, 1]` into ascending order?

- ○ **A.** 6 swaps
- ○ **B.** 3 swaps
- ○ **C.** 4 swaps
- ○ **D.** 10 swaps

Q19 (2 marks)

In merge sort, when the list `[7, 2, 5, 3]` is split completely into sublists, what are the resulting sublists?

- ○ **A.** [7, 2] and [5, 3]
- ○ **B.** [7], [2], [5], [3]
- ○ **C.** [7] and [2, 5, 3]
- ○ **D.** [2, 3, 5, 7]

Q20 (2 marks)

In binary search, if the current search range is from index 2 to index 8, which index is examined next?

- ○ **A.** Index 2
- ○ **B.** Index 6
- ○ **C.** Index 8

- ○ **D.** Index 5

Q21 (2 marks)

Why is binary search faster than linear search when searching large sorted datasets?

- ○ **A.** Because binary search uses more memory
- ○ **B.** Because each comparison eliminates approximately half of the remaining items
- ○ **C.** Because binary search starts at the end of the list
- ○ **D.** Because binary search checks multiple items simultaneously

Q22 (2 marks)

The list `[1, 2, 3, 4]` is already sorted. How can bubble sort detect this efficiently?

- ○ **A.** By checking if the first item equals the last item
- ○ **B.** By completing all possible passes regardless of the data
- ○ **C.** By only checking the middle item
- ○ **D.** By stopping when a complete pass makes no swaps

Q23 (2 marks)

In merge sort, when merging the sorted sublists `[2, 5]` and `[1, 6]`, what is the order in which items are added to the output list?

- ○ **A.** 2, 5, 1, 6
- ○ **B.** 1, 2, 6, 5
- ○ **C.** 2, 1, 5, 6
- ○ **D.** 1, 2, 5, 6

Q24 (3 marks)

A binary search is performed on the sorted list `[3, 7, 11, 15, 19, 23, 27]` to find the value 7. Trace the complete search, showing which items are compared at each step.

- ○ **A.** Compare 15 (not found), Compare 7 (found) - 2 comparisons total
- ○ **B.** Compare 3, Compare 7 (found) - 2 comparisons total
- ○ **C.** Compare 15 (too high, go left), Compare 7 (found) - 2 comparisons total
- ○ **D.** Compare 7 (found) - 1 comparison total

Q25 (3 marks)

Trace the complete bubble sort of the list `[5, 1, 4, 2]` into ascending order. How many complete passes are required?

- ○ **A.** 1 pass: [1, 4, 2, 5] - sorted
- ○ **B.** 3 passes: Pass 1 → [1, 4, 2, 5], Pass 2 → [1, 2, 4, 5], Pass 3 confirms sorted
- ○ **C.** 2 passes: Pass 1 → [1, 4, 2, 5], Pass 2 → [1, 2, 4, 5] - sorted

○ **D.** 4 passes required

Q26 (3 marks)

A teacher needs to sort class test scores. The list has 50 items. Compare the time efficiency of bubble sort and merge sort for this task.

○ **A.** Bubble sort is always faster than merge sort regardless of list size

○ **B.** Merge sort is generally faster for 50 items because it divides and conquers efficiently, while bubble sort may need to compare many pairs repeatedly across multiple passes

○ **C.** Both algorithms take exactly the same amount of time

○ **D.** Neither algorithm can sort 50 items

Q27 (3 marks)

Explain why binary search cannot work correctly on unsorted data, using the list `[8, 2, 9, 1, 5]` to search for the value 2 as an example.

○ **A.** Binary search would check position 0 first and miss the value 2

○ **B.** Binary search works fine on unsorted data; the sorted requirement is optional

○ **C.** Binary search would immediately find 2 at position 1

○ **D.** Binary search would find the middle item (9), determine 2 < 9, search the left half [8, 2], but following the algorithm's logic incorrectly assumes 2 must be left of 8 as well, potentially discarding the correct half

Q28 (3 marks)

Trace the complete merge sort process on the list `[6, 3, 8, 2]`, showing both the divide and merge phases.

- ○ **A.** Divide: [6, 3, 8, 2] → [6, 3] and [8, 2] → [6], [3], [8], [2]. Merge: [3, 6] and [2, 8] → [2, 3, 6, 8]
- ○ **B.** Split into [6, 3] and [8, 2], then merge to get [2, 3, 6, 8]
- ○ **C.** Sort left half [3, 6], sort right half [2, 8], combine as [3, 6, 2, 8]
- ○ **D.** Compare all items simultaneously and arrange as [2, 3, 6, 8]

Q29 (3 marks)

A library system stores 10,000 sorted book ISBNs. A librarian needs to check if a specific ISBN exists. Analyse why binary search is most suitable for this task.

- ○ **A.** Linear search would be fastest because it checks items in order
- ○ **B.** Binary search is ideal because the data is already sorted, and with 10,000 items, binary search needs only about 14 comparisons maximum while linear search might need up to 10,000 comparisons
- ○ **C.** Bubble sort should be applied first before any search
- ○ **D.** Merge sort would find the ISBN faster than any search algorithm

Q30 (3 marks)

A small online shop receives orders that need to be sorted by time for processing. Orders arrive continuously throughout the day. The shop receives approximately 20 orders per hour. Evaluate whether bubble sort or merge sort would be more suitable for maintaining the sorted order.

- ○ **A.** Merge sort is always better regardless of circumstances
- ○ **B.** Neither algorithm can handle continuous data
- ○ **C.** For this scenario with small batches arriving continuously, bubble sort may be practical because inserting new items into a nearly-sorted list requires few passes, while merge sort's division overhead is less advantageous for small additions to existing sorted data
- ○ **D.** Bubble sort cannot sort time-based data

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.